PURPLE INDUSTRIES · KUBERNETES LEARNER CONSOLE

Kubernetes

A hands-on CKA practice simulator

Inject faults, repair workloads, and get graded against real Kubernetes API behavior — no cloud account required.

purpleindustries.us/kubernetes

github.com/BarnsL/Kubernetes-Simulator

What is Kubernetes?

Kubernetes (K8s) is an open-source platform that automates the deployment, scaling, and operation of containerized applications across a cluster of machines.

You declare the desired state — Kubernetes makes it real

Tell the cluster "run 3 replicas of this image" and the control plane continuously reconciles reality toward that goal — restarting, rescheduling, and replacing as needed. A CNCF graduated project and the industry-standard container orchestrator.

Self-healing

Restarts, reschedules, and replaces unhealthy Pods automatically

Declarative

Desired state in YAML; controllers reconcile continuously

Elastic scaling

Scale workloads up or down on demand or automatically

Service discovery

Built-in DNS, networking, and load balancing for Pods

The core building blocks

Pod

Smallest deployable unit — one or more containers

Deployment

Declarative rollouts over self-healing ReplicaSets

Service

Stable network identity for a set of Pods

ConfigMap & Secret

Externalized configuration and sensitive data

PV & PVC

Persistent storage requests bound to volumes

RBAC

Roles and bindings that authorize every action

Namespace

Scoping and isolation boundaries in a cluster

Node

The worker machines that actually run Pods

THE PROJECT

Meet the CKA Practice Simulator

A Go CLI and web GUI for practicing Certified Kubernetes Administrator exam scenarios against a real or simulated cluster.

32 authentic mission labs

Pods, networking, storage, RBAC, scheduling & more

Inject → repair → grade loop

The same shape as real CKA exam tasks

Offline fake-client mode

Runs with no cluster and no cloud account

cka-sim — self-test

```
$ go build -o cka-sim ./cmd/cka-sim
$ ./cka-sim self-test

▶ scenario injected: demo-pod
image set to nginx:no-such-tag
status: Pending

✓ repair applied
image → nginx:1.25
✓ pod reached Running

RESULT: PASS
```

THE METHODOLOGY

How every mission works

1

INJECT

A fault is injected into the cluster

`image → nginx:no-such-tag`

2

INSPECT

Find the clue in events and spec

`kubectl describe pod`

3

REPAIR

Apply one targeted, scoped fix

`kubectl set image …`

4

GRADE

Spec + runtime checked, pass / fail

`phase = Running ✓`

You don't just read about Kubernetes — you break it, diagnose it, and fix it like an operator.

UNDER THE HOOD

Architecture

repo layout

```
cmd/cka-sim/ CLI entrypoint
internal/
grader/ pod state · image · readiness
injector/ broken images · misconfigs
scenarios/ scenario definitions
selftest/ end-to-end fake-client flow
web/ React + TypeScript GUI (Vite)

go test ./... all green
```

Separation of concerns

Scenario definitions live apart from grading logic

API is the source of truth

Validation reads real Kubernetes object state

Disposable-cluster first

Designed to run against throwaway local clusters

Offline by default

client-go/fake powers practice with no cluster

TWO WAYS TO RUN

Practice offline, or grade a real cluster

Fake-client mode

OFFLINE

Runs entirely offline using client-go/fake
No cluster, no kubeconfig, no cloud account
Deterministic, repeatable scenarios
Study anywhere — on a plane, offline

VS

Live mode

REAL CLUSTER

Grades against a real kubeconfig cluster
Use kind, minikube, or a full cluster
Builds real kubectl muscle memory
Closest to actual exam conditions

32 MISSION LABS

Coverage across the CKA curriculum

32 labs

Workloads & controllers

Pod image · Deployment rollout · readiness probe · CrashLoopBackOff · Job · CronJob · DaemonSet · StatefulSet

Config & storage

ConfigMap key · Secret env · resource requests · PV/PVC binding · finalizer troubleshooting

Networking & services

Service selector · Ingress backend · NetworkPolicy · EndpointSlice · DNS policy · cross-namespace lookup

Scheduling

Node scheduling clinic · taint & toleration match · label selector repair

Security & RBAC

RBAC binding · ServiceAccount binding · token mount control · Pod Security Baseline

Cluster setup & tooling

kubectl verify · kind bootstrap · minikube · Hello Minikube · kubeadm path · namespace cleanup

Pod image repair

BEGINNER

WORKLOADS

Identify a bad image value and recover a Pod to Running.

A Pod is stuck because its container image tag is invalid. The fix maps directly to one failing field — but a pass needs both the right image and a healthy runtime state.

BROKEN STATE
demo-pod 0/1 Pending · nginx:no-such-tag

inspect → repair → verify

```
$ kubectl describe pod demo-pod
Image: nginx:no-such-tag
Events: Failed to pull image

$ kubectl set image pod/demo-pod \
app=nginx:1.25
pod/demo-pod image updated

$ kubectl wait --for=condition=Ready \
pod/demo-pod
pod/demo-pod condition met ✓
```

PASS CRITERIA

`spec.image = nginx:1.25`

`status.phase = Running`

The web learning console

Learn Kubernetes concepts while you practice — terminal-first, with explanations at every step.

Simulated kubectl terminal

Run modeled commands safely against simulated state

Token-by-token breakdown

Every part of a command explained as you type it

Step-by-step tutorials

A guided inspect → repair → verify learning path

Mini cluster dashboard

Overview, workloads, services, config & storage, cluster

Glossary & field guide

Understand exactly what the grader is checking

Free-play sandbox

A mutable cluster for common kubectl verbs

A sandbox with real cluster state

Modeled kubectl verbs

`get` `describe` `create deployment` `run` `expose`

`set image` `scale` `label` `annotate` `taint`

`delete` `api-resources` `explain`

The simulated cluster has state. Later reads reflect earlier mutations, so create → scale → get behaves like a real session — not canned output.

Dashboard information architecture

- Overview
- Workloads
- Services
- Config & storage
- Cluster

Your latest command maps to the section that matters next.

QUICK START

Get started in minutes

Command-line (Go)

1. **Build the CLI**

```
go build -o cka-sim ./cmd/cka-sim
```

2. **Run the self-test**

```
./cka-sim self-test
```

3. **Inject, then grade**

```
./cka-sim inject … && ./cka-sim grade …
```

Web console

1. **Enter the web app**

```
cd web
```

2. **Install dependencies**

```
npm install
```

3. **Start the dev server**

```
npm run dev
```

Go 1.21+ | Node.js 18+ | kubectl (live mode) | MIT licensed

PURPLE INDUSTRIES

Break it. Fix it. Pass.

Hands-on Kubernetes practice that mirrors the real CKA exam — inject a fault, repair the workload, and let the grader confirm you got it right.

purpleindustries.us/kubernetes

github.com/BarnsL/Kubernetes-Simulator